



Unidade 1 — Back-End e Front-End



Objetivos de Aprendizagem

Ao final desta aula, o estudante deverá ser capaz de:

- **Compreender** o conceito de Back-End e seu papel na aplicação web;
- **Identificar** as principais tecnologias de Back-End (Node.js, PHP, Python, Java);
- **Explicar** o papel do banco de dados no armazenamento de dados;
- **Descrever** o ciclo de vida de uma requisição no servidor;
- **Comparar** as responsabilidades de Front-End e Back-End.

Taxonomia de Bloom: Compreender, Identificar, Explicar, Comparar.



Conceito de Back-End

- É a parte da aplicação que roda **no servidor**;
- Responsável pelo **processamento**, **armazenamento** e **segurança** dos dados;
- O usuário **não vê** diretamente o Back-End;
- É o "cérebro" da aplicação web.

Qual problema resolve? Processa dados, aplica regras de negócio, acessa bancos de dados e garante a segurança da aplicação.



Responsabilidades do Back-End

- **Processar** requisições HTTP recebidas do Front-End;
- **Executar** a lógica de negócio da aplicação;
- **Acessar** e **manipular** dados no banco de dados;
- **Autenticar** e **autorizar** usuários;
- **Garantir** a segurança dos dados;
- **Expor** APIs para consumo pelo Front-End;
- **Tratar** erros e exceções de forma adequada.



Tecnologias de Back-End

Tecnologia	Linguagem	Característica Principal
Node.js	JavaScript	Assíncrono, event-driven, mesma linguagem do Front-End
Express	JavaScript	Framework minimalista para Node.js
PHP	PHP	Amplamente utilizado, fácil hospedagem
Django	Python	Framework completo, "baterias incluídas"
Spring	Java	Robusto, enterprise, tipado
Laravel	PHP	Elegante, sintaxe expressiva



Banco de Dados



- **Banco de Dados:** sistema organizado para armazenar e recuperar dados;
- **Relacional (SQL):** MySQL, PostgreSQL, SQLite — dados em tabelas com relacionamentos;
- **Não-relacional (NoSQL):** MongoDB, Redis — dados em documentos, chave-valor, grafos.



Ciclo de Vida de uma Requisição

1. Usuário clica em "Salvar"
- ↓
2. Front-End envia POST /api/usuarios
- ↓
3. Servidor recebe a requisição
- ↓
4. Middleware processa (JSON, CORS, autenticação)
- ↓
5. Rota direciona para o Controller
- ↓
6. Controller executa a lógica de negócio
- ↓
7. Repository acessa o banco de dados
- ↓
8. Resposta é construída (JSON)
- ↓
9. Front-End recebe e atualiza a tela



Exemplo de Back-End com Express

```
const express = require('express');
const app = express();

app.use(express.json());

let usuarios = [];

// GET – Listar todos os usuários
app.get('/api/usuarios', (req, res) => {
  res.json(usuarios);
});

// POST – Criar um novo usuário
app.post('/api/usuarios', (req, res) => {
  const { nome, email } = req.body;
  const usuario = { id: usuarios.length + 1, nome, email };
  usuarios.push(usuario);
  res.status(201).json(usuario);
});

app.listen(3000, () => {
  console.log('Servidor rodando na porta 3000');
});
```





Erros Comuns

Erro	Consequência	Solução
Não validar dados no Back-End	Dados inválidos no banco	Sempre validar no servidor
Expor detalhes internos no erro	Falha de segurança	Mensagens genéricas para o cliente
Não tratar exceções	Servidor trava	Usar try/catch e middleware de erro
Confundir autenticação com autorização	Falha de segurança	Implementar ambos separadamente
Não usar HTTPS	Dados interceptados	Sempre usar HTTPS em produção



Síntese da Aula

Conceitos principais:

1. Back-End é a **camada de processamento** que roda no servidor;
2. É responsável pela **lógica de negócio**, **banco de dados** e **segurança**;
3. Tecnologias populares: **Node.js**, **Express**, **PHP**, **Django**, **Spring**;
4. O **banco de dados** garante a persistência dos dados;
5. O ciclo de vida de uma requisição envolve **múltiplas etapas** no servidor.

Pergunta reflexiva:

Se o Back-End fosse responsável apenas por servir arquivos estáticos, precisaríamos dele?

Próxima aula: Como Front-End e Back-End se comunicam? — Vamos estudar HTTP, APIs e o contrato entre as camadas.

